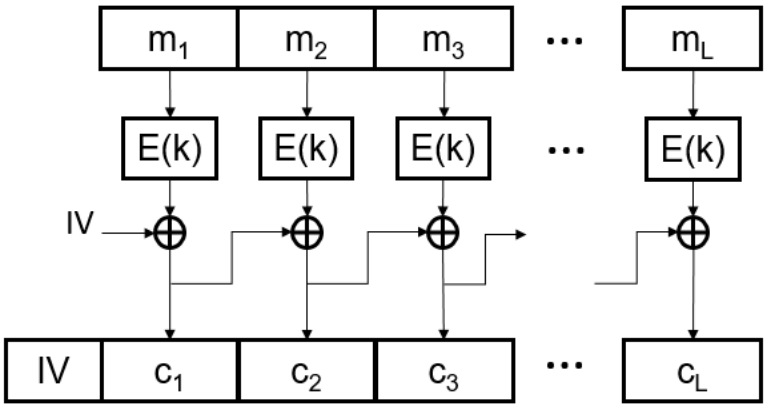
Câu 1. Các chế độ mật mã đối xứng

a. Sử dụng chế độ mật mã nào sau đây có thể chống lại được tấn công CPA?(Chọn tất cả đáp án đúng)

	Chế độ mã ECB (*Gợi ý: Không chống được KPA nên không chống được CPA*)
	Chế độ mã CBC sử dụng giá trị IV từ một bộ đếm (*Gợi ý: IV không ngẫu nhiên)*
x	Chế độ mã CBC sử dụng giá trị IV tạo ra từ thao tác truy cập ổ đĩa cứng
x	Chế độ mã CTR sử dụng giá trị IV từ một bộ đếm
	Chế độ mã CTR sử dụng giá trị IV là kích thước của thông điệp (*Gợi ý: IV không ngẫu nhiên*)

b. Bob thiết kế chế độ mã new-CBC như sau:

- Khởi tạo giá trị IV có kích thước n bit theo ngẫu nhiên phân bố đều.
- Khối m_1 có kích thước n bit được mã hóa: $c_1 = E(k, m_1)$ Å IV
- Mỗi khối m_i (với $i > 1$) có kích thước n bit được mã hóa: $c_i = E(k, m_i)$ Å c_{i-1}
- Giá trị IV được đính kèm với bản mã



Hãy vẽ sơ đồ giải mã.

c. Chế độ mã new-CBC có an toàn trước tấn công CPA không? Hãy chứng minh câu trả lời.

Gợi ý: Giải thích bằng mô hình tấn công thử thách

1. *Phía tấn công gửi hai bản tin kích thước 2 khối là a || a và a || b*
2. *Phía thử thách lựa chọn bản tin để mã:*

Nếu a || a được mã: $c_1 = IV \oplus E(k, a)$

$$c_2 = c_1 \oplus E(k, a) = IV \oplus E(k, a) \oplus E(k, a) = IV$$

Nếu a || b được mã: $c_1 = IV \oplus E(k, a)$

$$c_2 = c_1 \oplus E(k, b) = IV \oplus E(k, a) \oplus E(k, b) \neq IV$$

3. Khi nhận được c, phía tấn công kiểm tra:*

- *Nếu khối* $c_2 = IV$ *thì a || a đã được mã*
- *Ngược lại, thì a || b đã được mã*

Như vậy, xác suất đoán đúng là 1: new-CBC Không an toàn trước tấn công CPA

Câu 2. Xác thực thông điệp quảng bá

Giả sử server cần gửi thông điệp tới tất cả client C_1, C_2, .., C_n. Mỗi client khi nhận được thông điệp này cần xác minh được rằng thông điệp do chính server gửi. Giả sử rằng đã có một kênh an toàn được sử dụng để phân phối khóa giữa các bên.

d. Giả sử server và tất cả client chia sẻ một khóa đối xứng ngẫu nhiên cho từng phiên quảng bá. Server sử dụng khóa này để tạo mã MAC và gắn vào mỗi bản tin được gửi đi. Client có thể tin tưởng vào thông điệp nhận được là do server gửi và nội dung thông điệp toàn vẹn hay không? Giải thích.

Gợi ý: Không, vì tất cả client đều sở hữu khóa nên có thể giả mạo thông điệp của server

b. Giả sử server chia sẻ một khóa ngẫu nhiên với mỗi client cho từng phiên quảng bá. Server sử dụng khóa này để tạo mã MAC và gắn vào mỗi bản tin được gửi đi. Client có thể tin tưởng vào thông điệp nhận được là do server gửi hay không? Giải thích.

Gợi ý: Có, vì khóa chỉ được chia sẻ giữa client và server

e. Giả sử client nhận được một thông điệp từ server có chứa phần mềm độc hại. Client có thể khởi kiện server hay không? Giải thích.

Gợi ý: Không, vì mã MAC không cung cấp tính năng chống từ chối. Server có thể phủ nhận bản tin đã gửi.

Câu 3. Chữ ký số dựa trên RSA

Alice sử dụng chữ ký số RSA xây dựng theo 2 sơ đồ sau:
Sơ đồ đơn giản: Chữ ký số sig = $E(K_R, m)$
Sơ đồ tiêu chuẩn: Chữ ký số sig = $E(K_R, H(m))$
Trong đó:
$K_R = (d, n)$ là khóa cá nhân của Alice được giữ an toàn bí mật
$K_U = (e, n)$ là khóa công khai của Alice đã được công bố trên website tin cậy
H(m): Hàm băm mật mã an toàn
Bản tin được Alice gửi cho Bob là (m, sig).

a. Nếu Alice sử dụng chữ ký số đơn giản, bằng cách nào Bob có thể kiểm tra tính xác thực của bản tin.

Gợi ý: Bob giải mã bằng khóa công khai để được bản tin và so sánh bản tin sau khi giải mã với bản tin nhận được

b. David muốn tấn công vào tính xác thực của bản tin dùng chữ ký số đơn giản. Bằng cách nào David có thể tạo ra bản tin (m, sig) đánh lừa được Bob là bản tin này do Alice gửi.

Gợi ý: Chọn sig là một chuỗi ngẫu nhiên bất kỳ, sau đó tìm ra bản tin $m=D(K_U, sig)$

c. Cách thức tấn công trên của David có tác dụng không nếu Alice sử dụng chữ ký số tiêu chuẩn. Hãy giải thích.

Gợi ý: Với cách tấn công trên, kết quả giải mã là H(m), không thể xác định m vì hàm băm có tính chất một chiều

Câu 4. Xác thực danh tính bằng mật khẩu

Thông tin tài khoản người dùng được lưu trong cơ sở dữ liệu của server gồm định danh (userID) và giá trị băm của mật khẩu Hash(password). Giao thức xác thực giữa server và client như sau:
(1) Client → Server: userID || password

(2) Server băm mật khẩu nhận được từ yêu cầu của client và so sánh với giá trị đã lưu trong CSDL. Nếu trùng khớp thì xác thực thành công

a. Hãy mô tả 2 điểm yếu của hệ thống xác thực trên.

Gợi ý: Mật khẩu được gửi ở bản rõ

Giao thức có thể bị tấn công phát lại

b. Mô tả giải pháp để sữa chữa các điểm yếu đã chỉ ra.

Gợi ý: Giao thức trên là PAP, xem lại giao thức CHAP

Câu 5. Lỗ hổng phần mềm

Một số hàm chương trình được viết như sau:

```
1   typedef struct product{
2      char name[256];
3      short quantity;
4   } product;
5
6   /* Order products of the store
7   Return total quantity of products */
8   int order(product list[])
9   {
10     char tmp_name[256];
11     int n_products = 0;
12     while(1)
13     {
14        printf("Product: ");
15        gets_s(tmp_name, 255);
16        if(strlen(tmp_name) == 0) break;
17        strcpy(list[n_products].name, tmp_name);
18        printf("Quantity: ");
19        scanf("%d", &list[n_products].quantity);
20        n_products++;
21     }
22     return n_products;
23  }
24
25  /* Send shopping cart to server
26  Return number of transferred bytes, -1 if error */
27  int sendCart(int socket, product list[], int n)
28  {
29     char buff[4096], str_quantity[10];
30     int ret, len;
31     for (int i = 0; i < n, i++){
32        len = strlen(list[i].name);
33        strncat(buff, list[i].name, len);
34        strncat(buff, " ", 1);
35        sprintf(str_quantity, "%d\n", list[i].quantity);
36        len = strlen(str_quantity);
37        strncat(buff, str_quantity, len);
38     }
39     ret = send(socket, buff, strlen(buff), 0);
40     return ret;
```

41	}

a. Hãy chỉ ra 2 lỗ hổng phần mềm trong đoạn chương trình trên dẫn đến kẻ tấn công có thể khai thác để chèn mã độc thực thi.

Gợi ý:

- *Trong hàm order(), từ dòng 12 đến 21, thực hiện vòng lặp để thêm sản phẩm vào mảng list[] mà không giới hạn số phần tử thêm vào. Lỗ hổng này có thể dẫn đến tấn công tràn bộ đệm.*
  Lưu ý: `strcpy(list[n_products].name, tmp_name)` không gây ra lỗ hổng do lời gọi hàm gets_s(tmp_name, 255) đã giới hạn số ký tự tối đa ghi vào tmp_name là 255 ký tự. Khi đó kích thước của tmp_name tối đa là 256 ký tự, bao gồm ký tự kết thúc xâu, bằng với kích thước của trường name trong cấu trúc product.
- *Hàm sendCart(): Lệnh `strncat(buff, list[i].name, len);` tại dòng 33 và 37 không giới hạn kích thước xâu sau khi được nối thêm*

b. Viết lại mã nguồn cần thiết để sửa các lỗ hổng trên

- Vá lỗ hổng trên hàm order()

```
int order(product list[], unsigned int max)
{
   char tmp_name[256];
   int n_products = 0;
   while(n_products < max)
   {
      printf("Product: ");
      gets_s(tmp_name, 255);
      if(strlen(tmp_name) == 0) break;
      strcpy(list[n_products].name, tmp_name);
      printf("Quantity: ");
      scanf("%d", &list[n_products].quantity);
      n_products++;
   }
   return n_products;
}
```

- Vá lỗ hổng trên hàm sendCart()

```
if(4096 - strlen(buff) - strlen(list[i].name) - strlen(str_quantity) - 1 >=0){
     strncat(…);
}
```

Cách khác: Dùng hàm strlcat(char *dst, const char *src, unsigned int size)

size: Giới hạn kích thước tối đa của xâu sau khi nối, bao gồm cả ký tự '\0' kết thúc xâu.

```
for (int i = 0; i < n, i++){
      strlcat(buff, list[i].name, 4096);
      strncat(buff, " ", 4096);
      sprintf(str_quantity, "%d\n", list[i].quantity);
      strncat(buff, str_quantity, 4096);
   }
```

Phụ lục: Một số hàm C/C++

`char *gets_s(char *buffer, size_t n)`

Gets a line from the **stdin** stream.

Parameters

buffer: Storage location for input string.

n: The size of the buffer.

Return Value: Returns *buffer* if successful. A **NULL** pointer indicates an error.

`int sprintf(char *buffer, const char *format, [arguments] ...);`

Write formatted data to a string.

Parameters

buffer: Storage location for output

format: Format-control string

arguments: Optional arguments

Return Value: The number of bytes stored in *buffer*, not counting the terminating null character.

`char *strncat(char *strDest, const char *strSource, size_t count);`

The **strncat** function concatenates two string.

Parameters

strDest: Null-terminated destination string.

strSource: Null-terminated source string.

count: Number of characters to append.

Return Value: Returns a pointer to the destination string. No return value indicates an error.

`int send(int socket, char *buffer, size_t length, int flags);`

The **send** function sends data on a connected socket.

Parameters

socket: Specifies the socket file descriptor.

buffer: Points to the buffer containing the message to send.

length: Specifies the length of the message in bytes.

flags: Specifies the type of message transmission.

Return Value

Upon successful completion, *send*() shall return the number of bytes sent. Otherwise, -1 shall be returned and *errno* set to indicate the error.